

Predicting Box Office Success: A Machine Learning Approach to Movie Revenue Forecasting

Group 3

Department of Computer Science
University of North Carolina Greensboro
Greensboro, USA
mlcvach@uncg.edu

Abstract—The film industry is a very large industry. With production budgets sometimes reaching hundreds of millions of dollars, knowing whether a movie will make money before it releases is very important for studios. For our project, we chose to use the TMDB Movie Metadata dataset to try and predict box office revenue. We cleaned data, tested different hypotheses, and created five different machine learning models: Linear Regression, Decision Trees, Random Forests, Gradient Boosting, and K-Nearest Neighbors (KNN).

After evaluating each, we found the Random Forest Regressor performed best. It gave us an R^2 score of about 0.71, a noticeable improvement over our baseline linear model. When we looked at what features mattered most, production budget and vote count (how popular a movie is) were the biggest predictors. This report goes through the entire process, from figuring out the problem to validating our final model, to try to predict the revenue a movie will make.

Index Terms—Data Science, Machine Learning, Random Forest, Regression, Movie Revenue Prediction, TMDB Dataset

I. INTRODUCTION

The movie industry can be expensive. A movie could make billions, but it could also bankrupt a studio. As time passes, companies have more and more data they can use to make more educated choices.

Our goal for this project was to build a model that predicts a movie's revenue based on information that's generally available before release (like budget and genre), if not, shortly after (like audience popularity). Studios or investors could use a predictive model to see if a movie is worth the risk. We specifically wanted to see if we could beat a simple baseline prediction that just assumes "more money in = more money out."

A. Problem Framing

We treated this as a predictive problem, as we are trying to predict a specific number (revenue) rather than a category like "Success" or "Failure." We picked features like budget, genre, popularity, and vote counts because they feel like they should drive success. Budget shows the scale of the movie, while popularity shows how much people care about it.

B. Dataset

We used the **TMDB 5000 Movie Metadata** dataset from Kaggle. We picked this one because it included over 5,000 movies and included the following features:

- **Numerical:** Budget, Revenue, Runtime, Popularity, and Votes.
- **Categorical:** Genres, Production Companies, and Release Dates.

This gave us a chance to look at both the financial side (budget) and the social side (votes).

II. METHODOLOGY

Our project had four main stages - Framing, Exploration, Modeling, and Validation.

A. Data Preprocessing

Our data was a bit messy, so our first step was data cleaning. We found quite a few missing values in the 'runtime' and 'release_date' columns. We also dropped rows that didn't have a revenue or budget values, since we can't train a model without the answers.

One big issue we found in Stage 2 was the shape of the data. When we looked at the 'budget' and 'revenue' columns, they were heavily skewed (Skewness ≈ 2.22). Basically, most movies make a modest amount of money, but a few massive outliers (like *Avatar*) make billions. This "long tail" makes it hard for models to learn. To fix this, we applied a log transformation ($\log(1 + y)$) to the revenue and budget. This squashed the huge numbers down and made the data look more like a normal bell curve, which helped our models converge faster.

B. Exploratory Data Analysis (EDA)

Before we started modeling, we ran some statistical tests. A Chi-Squared test showed us that our numeric variables were definitely not normally distributed ($p < 0.05$). This told us that simple linear models might struggle, and that tree-based models (which don't care as much about normal distributions) might work better.

We also made a correlation matrix to check for relationships. As we expected, 'budget' and 'revenue' had a strong positive correlation (0.71).

C. Model Selection

We tested five different models to see which one would handle the data best:

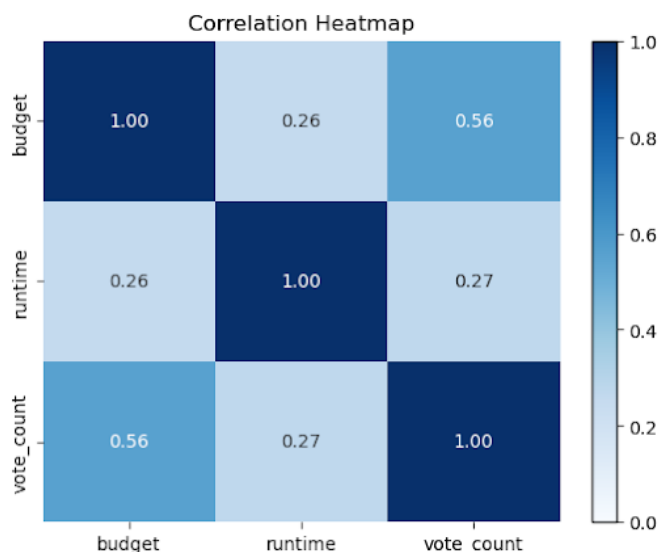


Fig. 1. Correlation Matrix from our Stage 2 analysis. You can see the high correlation between Budget and Revenue compared to Runtime.

- **Linear Regression:** Our baseline. If we couldn't beat this, we were doing something wrong.
- **Decision Tree:** Good for non-linear data, but tends to memorize the training set (overfit).
- **Random Forest:** A group of decision trees. We used 500 estimators to smooth out the errors.
- **Gradient Boosting:** Learns from its own mistakes iteratively.
- **K-Nearest Neighbors (KNN):** Predicts based on similar movies.

III. RESULTS AND DISCUSSION

A. Testing Our Hypotheses

Early on, we laid out three hypotheses to check if our intuition was right:

- 1) **Budget vs. Revenue:** We confirmed there is a strong positive link here ($p \approx 0.00$). Spending money helps you make money.
- 2) **Popularity vs. Revenue:** If people are talking about the movie online, it usually sells more tickets.
- 3) **Genre vs. Revenue:** We found that Adventure and Action movies make significantly more on average than genres like Drama.

B. Comparing the Models

Table I shows how everyone performed. We looked at R^2 (how much variance we explained) and RMSE (how far off our predictions were in dollars).

The **Random Forest Regressor** was the clear winner with an R^2 of 0.7126. It beat the Linear Regression baseline by about 4% and crushed the single Decision Tree ($R^2 = 0.46$). The Decision Tree likely overfitted—it memorized the training movies but couldn't predict new ones. Gradient Boosting was

close ($R^2 = 0.69$), but Random Forest was just a bit more stable.

C. Double Checking with Cross-Validation

We didn't want to just trust one test run, so in Stage 4 we did a **5-Fold Cross-Validation**. This splits the data 5 different ways to make sure we didn't just get lucky.

Our average R^2 was:

$$0.611 \pm 0.032$$

This is a bit lower than our single test score, which is normal, but the low standard deviation (± 0.032) is good news. It means our model is consistent.

D. What Actually Matters? (Feature Importance)

Since we used a Random Forest, we could look inside the model to see which features it was actually using to make decisions.

- **Vote Count** was actually the #1 predictor. This is interesting—it suggests that "hype" (number of people voting) matters even more than the budget.
- **Budget** was a close second.
- **Runtime** didn't really matter. A long movie doesn't guarantee a hit.

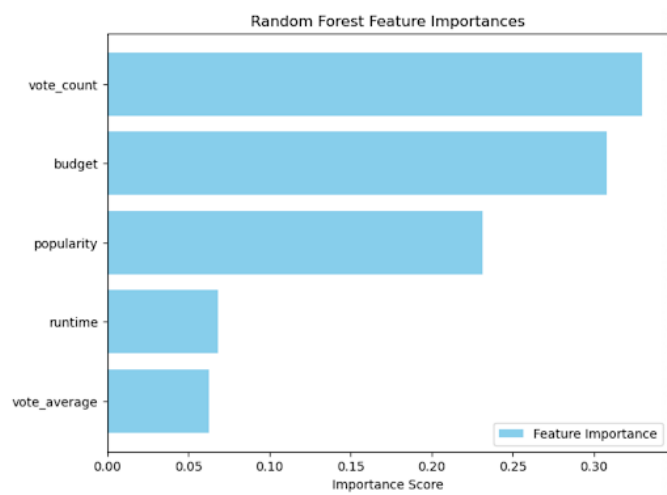


Fig. 2. Feature Importance from the Random Forest. Vote Count dominates, followed by Budget.

E. Visual Analysis

We plotted our Residuals (Predicted vs Actual). Ideally, we want these centered around zero. Our plot looked pretty good, though we did see some "fanning out". Basically, the more money a movie makes, the harder it is to predict the exact number. Predicting the difference between \$10k and \$20k is easy; predicting the difference between \$1.1B and \$1.2B is hard.

TABLE I
MODEL PERFORMANCE COMPARISON (STAGE 3 RESULTS)

Model	R^2 Score	RMSE (\$)	Notes
Linear Regression (Baseline)	0.6750	89,451,170	Simple but limited
Decision Tree	0.4684	114,486,913	Overfitted badly
Random Forest	0.7126	84,173,152	Best performance
Gradient Boosting	0.6927	87,039,088	Very close second
KNN Regressor	0.5778	102,026,939	Struggled with dimensions

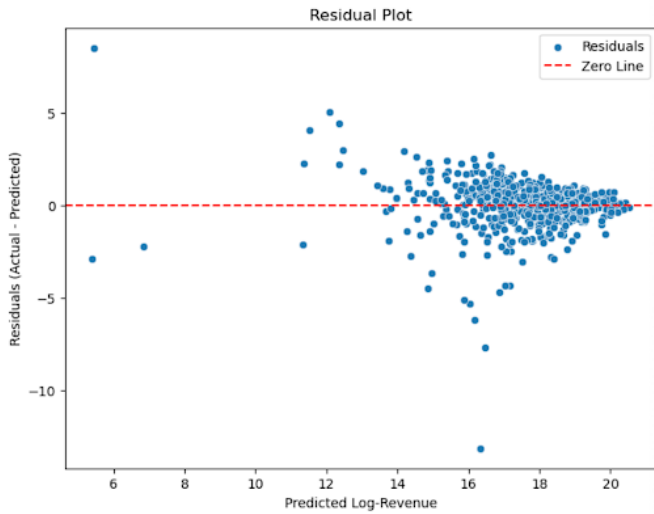


Fig. 3. Residual Plot. Errors are mostly centered around zero, which is what we want to see.

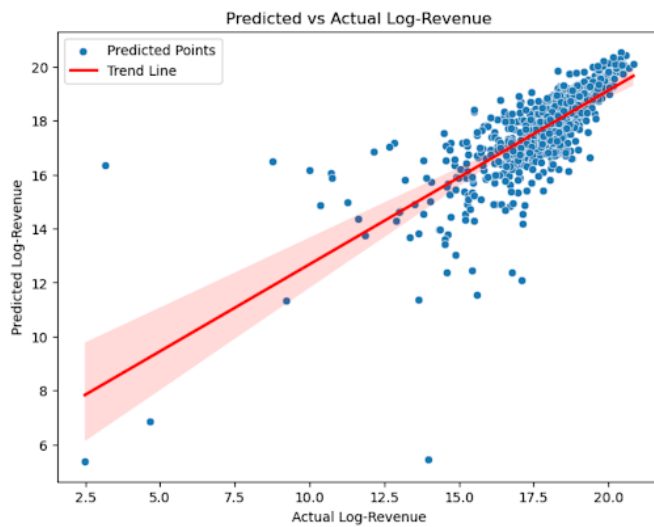


Fig. 4. Predicted vs Actual Log-Revenue. The points cluster around the line, showing our predictions are decent.

F. Discussion: The Human Element

In Stage 4, we built an interactive chart to look at our "misses." We found something interesting: the model struggled with "flops." If a movie had a massive budget, our model assumed it would make money. But movies like *The Lone Ranger* had huge budgets and tanked. The model couldn't predict that because it doesn't know the movie was bad; it just knows it was expensive. On the flip side, it missed low-budget breakouts because they break the "Budget = Success" rule. This shows that while data is powerful, there is a human element to art that numbers can't fully capture.

IV. CONCLUSION AND FUTURE WORK

Overall, we successfully built a pipeline that can predict movie revenue reasonably well. We started with raw data, cleaned it up, tested our assumptions, and found that a **Random Forest Regressor** was our best bet.

A. Key Takeaways

- **Money helps, but Hype is better:** Budget is important, but Audience Engagement (Vote Count) was actually the stronger signal in our model.
- **Ensembles work:** Simple models failed. We needed the complexity of a Random Forest to capture the nuance of the film industry.
- **Cleaning matters:** The log transformation we did in Stage 2 was crucial. Without it, the outliers would have ruined our training.

B. Future Work

If we had more time, we would love to try:

- 1) **Text Analysis:** Using NLP on the plot summaries to see if certain keywords (like "Superhero" or "Sequel") drive revenue.
- 2) **Cast Power:** Our model ignored the actors. Creating a "Star Power" score based on an actor's past box office hits would probably boost accuracy a lot.
- 3) **Timing:** Analyzing release dates (Summer vs Christmas) to catch seasonal trends.

REFERENCES

- [1] TMDb 5000 Movie Metadata Dataset, Kaggle. Available: <https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata>
- [2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [3] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.
- [4] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189-1232, 2001.